

The Architecture Synthesis Algorithm

A General Method for Approximating Architecture from Purpose

Macharia Barii

October 3, 2026

Abstract

We present an algorithm that, given a purpose or goal and sufficient information about available parts and environment, approximates the architecture that best serves that purpose. The algorithm extracts the five principles (function, speed/efficiency, safety/security, scale/capacity/load, aesthetics/beauty/symmetry) as functionals, discovers constraints by analysis and by probing/running the architecture, and iteratively refines the arrangement through a co-evolutionary loop. We formalize narrow mode (unique computable equilibrium) and extended mode (multiple or contested equilibria as an equilibrium-constrained problem), prove convergence conditions, analyze complexity, state the reduction theorem, and present theoretical guarantees. The algorithm is anytime, substrate-independent, and accommodates passive/active, living/non-living environmental elements.

Contents

1	Problem Statement	1
2	Architecture Synthesis: Narrow Mode	1
2.1	Overview	1
2.2	Algorithm	2
2.3	Subroutines	2
2.4	Convergence and Complexity (Narrow Mode)	3
3	Extended Mode: Analysis of Games and Relations	3
3.1	Architecture as Mechanism	3
3.2	Equilibrium-Constrained Design	3
3.3	Extended Algorithm	4
3.4	Conflict Traps and Equilibrium Classification	4
4	Theoretical Guarantees	4
5	Reduction, Turing Barrier, and Practical Considerations	5
5.1	Reduction Theorem	5
5.2	Turing Barrier	5
5.3	Practical Considerations	5
6	Conclusion	5

1 Problem Statement

Definition 1.1 (Input). The algorithm receives: (i) a purpose/goal as utility $u : O \rightarrow \mathbb{R}$ or goal set $G \subseteq O$; (ii) a universe of parts $\mathfrak{P}$ with attribute spaces $\mathcal{A}_p$; (iii) environment $E = E_P \sqcup E_A$

(passive/active); (iv) budget B (time, cost, compute); (v) oracle/simulator $\varphi : \mathcal{A}(P) \times E \rightarrow O$.

Definition 1.2 (Output). Architecture $A^* = (P^*, R^*, \alpha^*)$ such that

$$A^* \approx \arg \max_{A \in \Phi(P, E, w)} U(\pi(A); w), \quad (1)$$

where $\pi = (F, \sigma, \Sigma, K, -\rho)$ are the five principles (function, speed/efficiency, safety/security, scale/capacity/load, aesthetics/beauty/symmetry) and Φ is the emergent feasible set.

Remark 1.3. Φ is not known in advance; it must be discovered while searching.

2 Architecture Synthesis: Narrow Mode

2.1 Overview

Two nested loops: outer (architecture refinement: select parts, propose arrangements, evaluate, adjust) and inner (constraint discovery: run/probe, observe failures/bottlenecks, update $\hat{\Phi}$). Alternates exploration and exploitation; terminates when stable or budget exhausted.

2.2 Algorithm

2.3 Subroutines

formalize(goal) Convert goal to $u : O \rightarrow \mathbb{R}$ (and optionally G); encode hard constraints if applicable.

extract_principles(u, E) Derive measurable functionals $(F, \sigma, \rho, \Sigma, K)$ from purpose and environment.

select_initial_parts($\mathfrak{P}, u, E$) Relevance/diversity/affordability filter; seed parts maximizing expected F .

generate_probes(A, E) Load, stress, adversarial, boundary, and active-element probes.

infer_constraint(probe, outcome) Infer constraint $c(A; P, E, w) \leq 0$ (from parts/environment/principles/int

adjust_parts(P, $\hat{\Phi}$, π , w) Add/remove/swap parts to satisfy $\hat{\Phi}$ while improving π .

optimize_arrangement(P, $\hat{\Phi}$, π , w) Search over $\mathcal{A}(P) \cap \hat{\Phi}$; use heuristics (e.g. GA, SA) or multi-objective (NSGA-II/MOEA-D) for Pareto approximation.

measure(A, E) Evaluate $(F, \sigma, \Sigma, K, -\rho)$ via run/simulate and measurement.

environment_response(A, E) Apply r_e for active $e \in E_A$, update parameters, reclassify if status changes.

reweight(π , u , E) Adjust weights w in response to violations/environment shifts.

2.4 Convergence and Complexity (Narrow Mode)

Assumption 2.1 (Bounded utility). U is bounded above by $U_{\max} < \infty$.

Assumption 2.2 (Monotone improvement). Each outer iteration does not decrease U (modulo exploration).

Assumption 2.3 (Finite constraints). Φ is defined by finitely many constraints.

Assumption 2.4 (Discoverability). Every constraint in Φ is discoverable by some finite probe sequence.

Algorithm 1 ARCHITECT (narrow mode)

Require: goal u or G ; parts $\mathfrak{P}$; environment E ; budget B

Ensure: A^* , $\hat{\Phi}$

```
1:  $u \leftarrow \text{formalize}(\text{goal})$ 
2:  $\pi \leftarrow \text{extract\_principles}(u, E)$ 
3:  $w \leftarrow \text{initialize\_weights}(\pi)$ 
4:  $P \leftarrow \text{select\_initial\_parts}(\mathfrak{P}, u, E)$ 
5:  $\hat{\Phi} \leftarrow \emptyset$ 
6:  $A \leftarrow \text{initial\_arrangement}(P)$ 
7:  $t \leftarrow 0$ ,  $U_{\text{prev}} \leftarrow -\infty$ 
8: while  $\text{budget\_remaining}(B)$  and not  $\text{converged}(A, \hat{\Phi})$  do
    

---


    9:    $\text{probes} \leftarrow \text{generate\_probes}(A, E)$ 
    10:  for  $\text{probe} \in \text{probes}$  do
    11:     $\text{outcome} \leftarrow \text{run\_or\_simulate}(\text{probe}, E)$ 
    12:    if  $\text{violates\_expectation}(\text{outcome})$  then
    13:       $c \leftarrow \text{infer\_constraint}(\text{probe}, \text{outcome})$ 
    14:       $\hat{\Phi} \leftarrow \hat{\Phi} \cup \{c\}$ 
    15:    end if
    16:  end for
    

---


    17:   $P \leftarrow \text{adjust\_parts}(P, \hat{\Phi}, \pi, w)$ 
    18:   $A \leftarrow \text{optimize\_arrangement}(P, \hat{\Phi}, \pi, w)$ 
    

---


    19:   $\pi(A) \leftarrow \text{measure}(A, E)$ 
    20:   $U \leftarrow \text{scalarize}(\pi(A), w)$ 
    

---


    21:   $E \leftarrow \text{environment\_response}(A, E)$ 
    22:   $w \leftarrow \text{reweight}(\pi, u, E)$ 
    23:  if  $|U - U_{\text{prev}}| < \varepsilon$  and  $\hat{\Phi}$  stable then break
    24:  end if
    25:   $U_{\text{prev}} \leftarrow U$ ,  $t \leftarrow t + 1$ 
  26: end while
27: return  $A, \hat{\Phi}$ 
```

Proposition 2.5 (Convergence). *Under ?? 2.1–2.4, the algorithm converges to a local optimum of U over Φ in finite time.*

Remark 2.6. With active elements, Φ may be a moving target; algorithm converges to a moving equilibrium or limit cycle. The algorithm is *anytime*: returns usable A_t at any iteration with non-decreasing utility (modulo exploration).

3 Extended Mode: Analysis of Games and Relations

3.1 Architecture as Mechanism

Architecture A induces game $\mathcal{G}(A, E) = (I, \{S_i(A)\}_i, \varphi_A, \{u_i\}_i, \{\mu_i\}_i)$. Nash equilibrium $s^* = (s_1^*, \dots, s_N^*)$: $u_i(\varphi_A(s^*, E), \theta) \geq u_i(\varphi_A(s_i, s_{-i}^*, E), \theta)$ for all i, s_i . Equilibrium set $\mathcal{E}(A, E)$.

Phase	Complexity
Formalize goal	$O(1)$
Extract principles	$O(E)$
Select initial parts	$O(\mathfrak{P})$
Generate probes	$O(2^{ P })$ worst; $O(P k)$ heuristic
Run/simulate	$O(\text{sim})/\text{probe}$
Infer constraints	$O(\hat{\Phi})$
Optimize arrangement	NP-hard; heuristic $O(\text{iter} \cdot P ^2)$
Measure	$O(\text{deploy})$
Co-evolutionary update	$O(E_A)$
Total	$O(T \cdot (\text{probes} \cdot \text{sim} + \text{opt}))$

Table 1: Complexity by phase (T outer iterations).

3.2 Equilibrium-Constrained Design

Design becomes a Mathematical Program with Equilibrium Constraints (MPEC):

$$\max_{A \in \Phi} U(\varphi_A(s^*(A), E); w) \quad \text{s.t.} \quad s^*(A) \in \mathcal{E}(A, E). \quad (2)$$

3.3 Extended Algorithm

Algorithm 2 ARCHITECT_EXTENDED

Require: goal; parts $\mathfrak{P}$; environment E ; agents I ; budget B

Ensure: A^* or map, $\mathcal{E}^*(A^*)$, mode

```

1: mode  $\leftarrow$  select_mode(goal,  $I$ , info)
2: if mode = finding then return ARCHITECT(goal,  $\mathfrak{P}$ ,  $E$ ,  $B$ )
3: end if
4: for each  $i \in I$  do
5:    $u_i \leftarrow$  elicit_or_infer_utility( $i$ ,  $E$ )
6:    $S_i \leftarrow$  identify_strategy_space( $i$ ,  $A$ )
7:    $\mu_i \leftarrow$  infer_beliefs( $i$ ,  $E$ )
8: end for
9:  $\mathcal{E} \leftarrow$  compute_equilibria( $\{u_i\}$ ,  $\{S_i\}$ ,  $\{\mu_i\}$ ,  $E$ )
10: for each  $s^* \in \mathcal{E}$  do
11:   classify( $s^*$ )  $\in$  {peace, trap, fragile, chaos}
12:   basin( $s^*$ )  $\leftarrow$  compute_basin( $s^*$ )
13:   stability( $s^*$ )  $\leftarrow$  compute_stability( $s^*$ )
14: end for
15: for each candidate  $A' \in \mathcal{A}$  do
16:    $\mathcal{E}' \leftarrow$  compute_equilibria( $\mathcal{G}(A', E)$ )
17:   if  $\mathcal{E}'$  contains peace equilibrium with large basin then record  $A'$ 
18:   end if
19: end for
20: return best candidate  $A^*$ ,  $\mathcal{E}^*(A^*)$ , mode

```

Remark 3.1. In extended mode, output is a structured map (agents/utilities, $\mathcal{E}$, classification, basins, candidate interventions, uncertainty)—not a single guaranteed target.

3.4 Conflict Traps and Equilibrium Classification

Define: *Peace equilibrium* (A, s^*) with $s^* \in \mathcal{E}$, $\varphi_A(s^*, E) \in G$. *Conflict trap* (A, s^*) with $s^* \in \mathcal{E}$, $\varphi_A(s^*, E) \notin G$, > 0 , robust. *Fragile peace*: desirable but $s^* \notin \mathcal{E}$. Stability–desirability matrix yields four classes: peace, fragile, trap, chaos/collapse.

4 Theoretical Guarantees

Theorem 4.1 (Soundness). *If inference rules are sound, every constraint in $\hat{\Phi}$ corresponds to a real constraint in Φ .*

Theorem 4.2 (Completeness). *If every constraint in Φ is discoverable by some finite probe sequence, then $\hat{\Phi} \rightarrow \Phi$ as $t \rightarrow \infty$.*

Theorem 4.3 (Convergence). *Under ?? 2.1–2.4, the algorithm converges to a local optimum of U over Φ in finite time.*

Theorem 4.4 (Anytime property). *At any iteration t , A_t is feasible w.r.t. $\hat{\Phi}_t$ with utility U_t ; U_t is non-decreasing in t (modulo exploration).*

Theorem 4.5 (Co-evolutionary stability). *If the co-evolutionary system has a stable fixed point (A^*, E^*) and adaptation operator D is a contraction, the algorithm converges to (A^*, E^*) .*

5 Reduction, Turing Barrier, and Practical Considerations

5.1 Reduction Theorem

Theorem 5.1 (Reduction). *If $N = 1$ or all agents have identical utilities and the game has a dominant-strategy equilibrium, extended mode reduces to narrow mode.*

5.2 Turing Barrier

Theorem 5.2 (Turing barrier). *No algorithm can decide for arbitrary architecture with Turing-complete agents whether $\mathcal{E}(A, E) \neq \emptyset$, whether $\mathcal{E}(A, E)$ contains a peace equilibrium, or whether (A_t, E_t) converges.*

Theorem 5.3 (Long-Run Failure). *Any architecture relying on containment to secure compliance will eventually fail because a Turing-complete agent can find a strategy reducing effective cost of deviation below gain; agency cannot be permanently suppressed.*

5.3 Practical Considerations

- **Simulation impossible:** use digital twins, analogy, expert elicitation, formal verification.
- **Active elements unpredictable:** use robust/stochastic/adversarial optimization, learning.
- **Constraints unknown:** use exploration, failure injection, red-teaming, Bayesian inference.
- **Goal ambiguous:** use preference elicitation, inverse reinforcement learning, multi-objective frontier, scenario planning.
- **Environment classification:** treat passive/active, living/non-living carefully (misclassification costly).

6 Conclusion

The Architecture Synthesis Algorithm discovers constraints and iteratively refines arrangements to serve a stated purpose. In narrow mode (unique computable equilibrium), it converges to a locally optimal feasible architecture under stated conditions. In extended mode (multiple/contested equilibria), it produces a structured map of equilibria, classifications, basins, and candidate interventions rather than a single guaranteed solution. The algorithm is anytime, substrate-independent, handles passive/active and living/non-living elements, and respects co-evolution. Reduction and Turing barrier results delineate its scope: when games are trivial it collapses to optimization; when agents are Turing-complete, decision procedures are impossible.